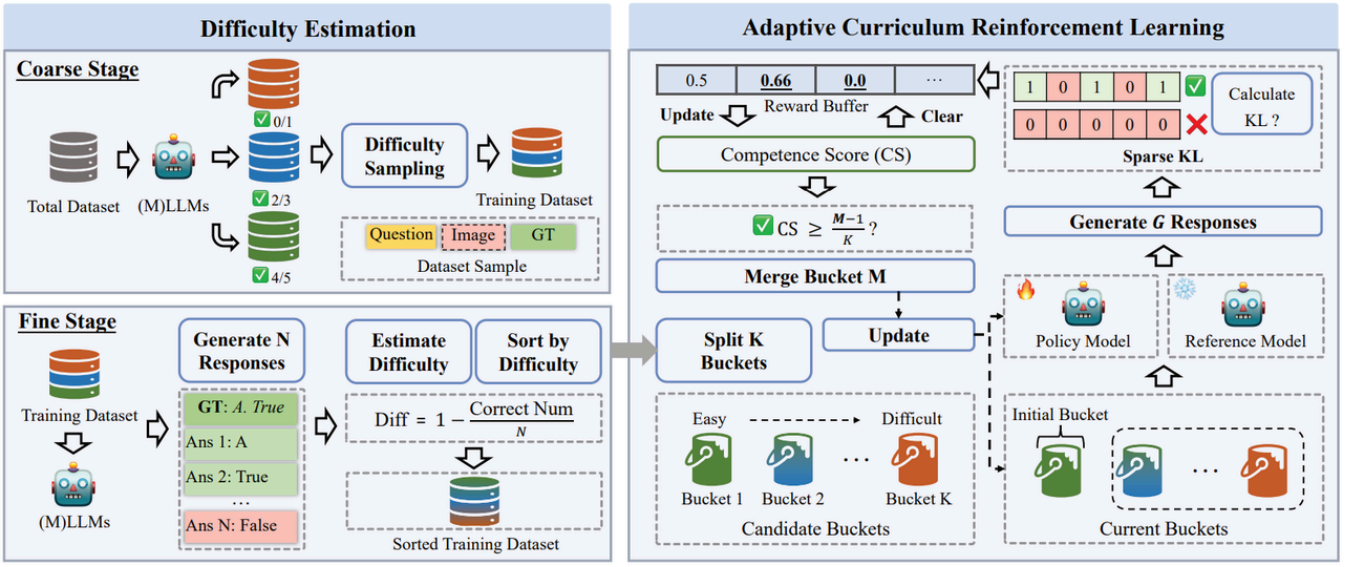
AdaCuRL: Adaptive Curriculum Reinforcement Learning with Invalid Sample Mitigation and Historical Revisiting



- GRPO may face two fatal problems:
  - When all answers in a group have the same reward, the train signal may be trivial.
  - In this case, the KL regulation term will dominate the loss. The model will be rollbacked to the old version.
  - A "wrapper" of GRPO can solve these problems.
- Concepts
  - Curriculum Learning: a technique inspired by human learning. Given a dataset, the model will learn knowledge from easy to hard to achieve better performance and faster convergence. Formally, given dataset D:

    $$D = \{(x_i, y_i, d_i)\}|_{i=1}^{N}, \text{and the difficulties, } d_i, \text{ is ordered increasingly.}$$
  - Self-Paced Learning: we assume that loss can directly reflect the difficulties of problems. We use loss as a measure of difficulty and set a threshold to limit the difficulty of the set. Formally:

$$D = \{(x_i, y_i)|l(\hat{y}, y) \leq \tau\}, \text{where } \tau \text{ is the threshold of the selection.}$$

  - That is to say, only QA pairs with loss less than τcan be added to the dataset.
  - We will dynamically increase or decrease τbased on the model's performance
- Coarse-to-Fine Difficulty Estimation in AdaCuRL
  - Coarse:
    - Given a dataset, let the model produce *five* responses for each question. Count the number of correct responses. Let $c$be the number of correct answers, $c_i \in \{0, 1, 2, 3, 4, 5\}$.
    - Create three bins to hold the questions with the corresponding # of correct responses:

$G_1 = \{i | c_i \in [0, 1]\}$
$G_2 = \{i | c_i \in [2, 3]\}$
$G_3 = \{i | c_i \in [4, 5]\}$

- Sample from the three bins:

$S = \bigcup_{k=1}^{3} \text{Sample}(G_k, \lfloor \rho_k |G_k| \rfloor)$, where ρ_k is the predefined sampling ratio for k-th bin.

- Fine:
  - For each question in S, generate Nanswers, where $N >> 5$. Let $c(q)$be the # of correct of each question, $\text{difficulty}(q) = 1 - \frac{c(q)}{N}$.
  - Filter out the question with difficulty higher than 0.95 or less than 0.05 to ensure the stability of the train. Sorted by difficulties, the result dataset is D.

- Curriculum Reinforcement Learning in AdaCuRL
  - We partition Dinto Kbuckets: $B_k|_{k=1}^{K}$with equal size but increasing difficulty.
  - At the beginning, we use B_1as the trainset D_c and gradually merge the following buckets into D_c .
    - To determine whether we update $D_c$or not, we maintain a csvariable, initialized to 0, to indicate the competence score $\in [0, 1]$.
    - The update rule is:

    $cs_{t+1} = cs_t + (\bar{r} - 0.5) \max(1 - cs_t, \gamma)$, where

    $\bar{r}$is the average reward.

    $\gamma$is a hyperparameter to avoid the rate being too small.

    When csreaches a ratio: $cs \geq \frac{k-1}{K}$, we update D_c. This works as $k$is related to the difficulty: larger k= higher difficulty.

    - *Intuitively, when the model cannot yet fully understand current D_c, its $\bar{r}$will be small (can be less than 0.5), so the framework will force the model to spend more time on questions from current D_c. Once it is mastered, $\bar{r}$is large, and $\bar{r} - 0.5$ will be positive, it increases csuntil it reaches the threshold.*
    - After merging, we reset $cs = \frac{k-1}{K}$.
  - Reward Function: *(This framework is for binary tasks)* The reward has two parts: format reward and accuracy reward.
    - The reward **serves two purposes**:
      1. Core GRPO update (group reward -> average -> calculate advantage)
      2. Decide when to update the current trainset D_c.
  - GRPO-related modification
    - Conditional KL to avoid model degrading:

    $L_{GRPO} = -\mathbb{E}_i[\min(\rho_i \cdot \hat{A}_i,\ \text{CLIP} \cdot \hat{A}_i)] + \beta \mathbf{I}[\hat{\mathbf{A}}_\mathbf{i} \neq \mathbf{0}] \mathbb{E}_i[KL(\cdot)]$

    - It's to say, when the advantage is 0, we don't calculate KL to keep the model untouched.

- Self-pacing Mechanism
  - We update Dafter some training.
  - Use the **updated model** to re-evaluate the difficulty of the problems and eliminate those being "too easy".

- $D' = \{q \in D | \text{new difficulty}_q \geq \tau'\}$
- Re-divide D' to buckets and continue the training.

- Algorithm Pseudocodes:

Algorithm 1: Curriculum-Based Reinforcement Learning

Input: Initial policy model π_θ, sorted dataset $\mathcal{D}$, number of buckets K, format reward cutoff step T_f, training step $T \leftarrow 0$, decay factor γ, competence score $cs \leftarrow 0$, reward accumulation length M
Init: Curriculum set $\mathcal{D}_c \leftarrow \emptyset$, reward buffer $\mathcal{R}_b \leftarrow \emptyset$
Partition $\mathcal{D}$ into equal buckets $\{\mathcal{B}_1, \mathcal{B}_2, ..., \mathcal{B}_K\}$
$\mathcal{D}_c \leftarrow \mathcal{B}_1$
while $\mathcal{B}_K \not\subset \mathcal{D}_c$ and samples in $\mathcal{B}_K$ not fully trained **do**
  Sample batch $X \sim \mathcal{D}_c$
  Generate $G \leftarrow \pi_\theta(X)$
  if $T < T_f$ **then**
    $R \leftarrow \text{Acc}(X, G) + \text{Format}(X, G)$
  else
    $R \leftarrow \text{Acc}(X, G)$
  end if
  Update policy π_θ using Eq. (3)
  Append R to $\mathcal{R}_s$
  if $|\mathcal{R}_s| \geq M$ and $T \geq T_f$ **then**
    Compute $\bar{r} \leftarrow$ average of $\mathcal{R}_s$
    Update competence score:

$$cs \leftarrow cs + (\bar{r} - 0.5) \cdot \max(1 - cs, \gamma)$$

    if $g \geq \frac{k-1}{K}$ **then**
      $\mathcal{D}_c \leftarrow \mathcal{D}_c \cup \mathcal{B}_k$, reset $\mathcal{R}_s$, set $cs \leftarrow \frac{k-1}{K}$
    end if
  end if
  Check whether R is all 0 or 1 (i.e., whether $\hat{A}_i$ is a zero vector), and compute the GRPO loss as:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_i[\rho_i \hat{A}_i] + \mathbb{I}[\hat{A}_i \neq 0]\, \beta\, \mathbb{E}_i\Big[\text{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})\Big]$$

  $T \leftarrow T + 1$
end while

- Workcited
    - Paper
    - Chat History